

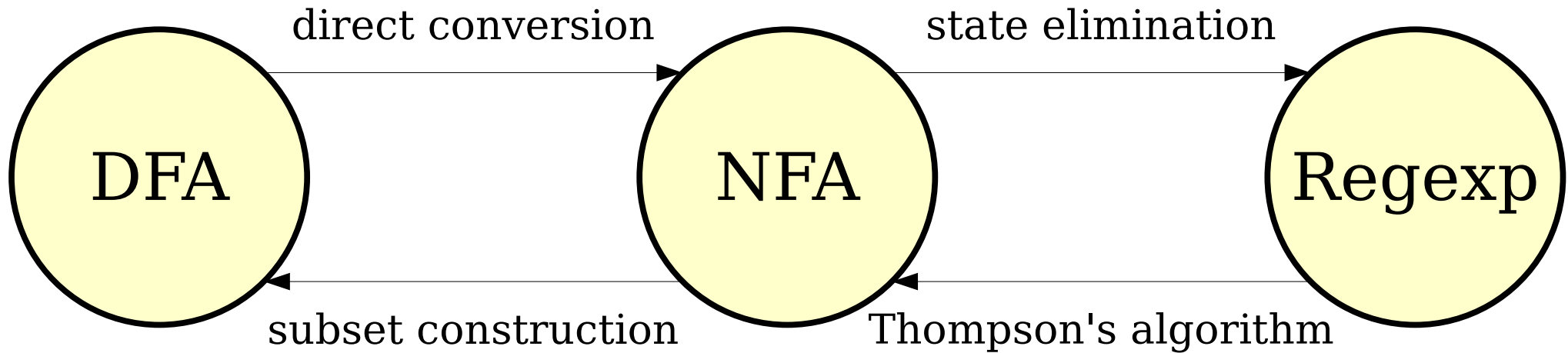
The State-Elimination Algorithm

- Start with an NFA N for the language L .
- Add a new start state q_s and accept state q_f to the NFA.
 - Add an ε -transition from q_s to the old start state of N .
 - Add ε -transitions from each accepting state of N to q_f , then mark them as not accepting.
- Repeatedly remove states other than q_s and q_f from the NFA by “shortcutting” them until only two states remain: q_s and q_f .
- The transition from q_s to q_f is then a regular expression for the NFA.

The State-Elimination Algorithm

- To eliminate a state q from the automaton, do the following for each pair of states q_0 and q_1 , where there's a transition from q_0 into q and a transition from q into q_1 :
 - Let R_{in} be the regex on the transition from q_0 to q .
 - Let R_{out} be the regex on the transition from q to q_1 .
 - If there is a regular expression R_{stay} on a transition from q to itself, add a new transition from q_0 to q_1 labeled $((R_{in})(R_{stay})^*(R_{out}))$.
 - If there isn't, add a new transition from q_0 to q_1 labeled $((R_{in})(R_{out}))$
- If a pair of states has multiple transitions between them labeled $R_1, R_2, \dots, R_k$, replace them with a single transition labeled $R_1 \cup R_2 \cup \dots \cup R_k$.

Our Transformations



Theorem: The following are all equivalent:

- L is a regular language.
- There is a DFA D such that $\mathcal{L}(D) = L$.
- There is an NFA N such that $\mathcal{L}(N) = L$.
- There is a regular expression R such that $\mathcal{L}(R) = L$.

Why This Matters

- The equivalence of regular expressions and finite automata has practical relevance.
 - Regular expression matchers have all the power available to them of DFAs and NFAs.
- This also is hugely theoretically significant: the regular languages can be assembled “from scratch” using a small number of operations!